

The 9% Challenge: Smart Customer Targeting

Personal Loan Campaign

Austin University – Post Graduate Program in Artificial Intelligence

01/17/2026

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model Performance Summary
- Appendix

Executive Summary

The Goal

Improve the bank's low **9% personal loan conversion rate** by identifying high-potential borrowers and reducing wasted marketing spend.

The Solution

We developed a **Pre-Pruned Decision Tree** model that successfully isolated the "Borrower" profile. The analysis revealed that **Annual Income, Family, and Education Level** are the dominant predictors, producing a distinct signal that is easy to detect.

Key Results

- **93% Recall:** The model correctly identifies **93 out of every 100** actual borrowers.
- **93% Precision:** When the model targets a customer, it is correct **93% of the time**.

Recommendation

Deploy the model immediately to shift from mass marketing to targeted "Sniper" campaigns. By focusing exclusively on high-income professionals, the bank can achieve near-perfect efficiency while capturing the vast majority of the market opportunity.

Business Problem Overview and Solution Approach

Overview

AllLife Bank has a strong liability base (depositors), but a disproportionately small asset base (borrowers). We are paying interest, but not maximizing earnings. Previous broad-spectrum marketing campaigns achieved a **9% conversion rate**. While healthy, this approach is resource-intensive and inefficient.

Approach

We want to shift from "Mass Marketing" to "Precision Targeting" by;

- Identify high-propensity customers within the existing depositor base.
- Reduce marketing spend by excluding low-probability targets.
- Exceed the 9% benchmark using predictive modeling.

Methodology

- Applied Exploratory Data Analysis(EDA) to decode the operational dataset.
- Applied model pruning to improve the result of the model.

Techniques

- **Classification Tree**

The Loan Modeling data features a streamlined structure that presents minimal data conversion challenges. This is particularly advantageous when using decision trees, as the algorithm operates directly on numeric representations, bypassing the need to encode non-numeric (categorical) features into a specific format before training.

The **Zip Code** feature is challenging. Although it is numeric, it serves as a nominal reference for location (region, city, post office). If the model interprets these numbers as mathematical values rather than categories, it may produce misleading results.

The **Experience** feature contains distinct data anomalies, specifically negative integers like -1, -2, and -3. These are invalid entries that misrepresent the customers' professional history and require cleaning.

Education is also a reference, but because it is a number crescent where higher values mean higher education It is usable as it is.

Personal_Loan, Securities_Account, CD_Account, Online, and CreditCard are Boolean features efficient to split the group, sometimes they can skew the results due to sparsity.

In particular, I missed the customer risk factor(Credit Score) and the length of the relationship with the Bank.

Data Preprocessing

- Duplicate value check

There is no duplicate customer data.

- Duplicate value check

There is no duplicate customer data.

- Missing value treatment

There are no missing values or null values in the dataset.

There is a hidden missing value in the Experience feature; it contains values such as (-1,-2,-3). The average age of the hidden missing feature information is 24.5

Data Preprocessing - Feature Engineering

- ZIP Code

It is converted to a category item, which provides richer information when describing the features, such as Unique and Top. E.g., Top shows the most frequent zipcode in the dataset 94720, before slicing it. This is more important than the avg of the zipcode.

The zipcode was reduced to the first 2 columns only to address the region, keep geography, and reduce overfitting. The zip code was too granular, so we reduced it to regions.

It will reduce model complexity in the future.

- Boolean Columns (Education, Personal_Loan, Securities_Account, CD_Account, Online, CreditCard)

They are converted from int64 to category, allowing Python to treat them as labels rather than numbers with a mathematical relationship. They are properly plotted by plotting libraries

Data Preprocessing - Outlier check (treatment if needed)

We used only the numeric columns to identify outliers; avoid treating labels as mathematical values.

We identify the outlier by calculating the lower and upper bounds for each column. Anything below the lower limit or above the upper limit is tagged as an outlier.

Some interesting findings

- 5.8% of customers have a mortgage above the \$252.5 thousands.
- Any income above \$186.5k is an outlier, 1.92% of the customers fall into this category
- 6.48% of the customers have a credit card debt above \$5.2k.

	Lower Bound	Upper Bound
Age	5	85
Experience	-20	60
Income	-49.5	186.5
Family	-2	6
CCAvg	-2	5.2
Mortgage	-151.5	252.5

Age	0
Experience	0
Income	1.92
Family	0
CCAvg	6.48
Mortgage	5.82

Data Preprocessing - Data preprocessing for modeling

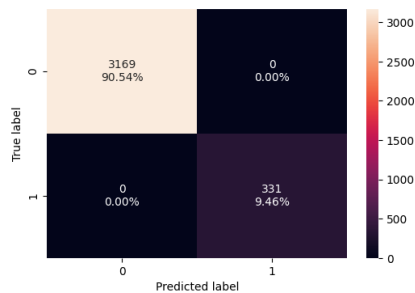
- Given that Personal Loan and Experience were correlated, they were dropped. They have a 99.4% correlation. A Personal Loan is used as a result prediction validator.
- We split the data set after hot encoding the Zip code and Education. The result will be 17 columns for the model. Will only have Education_2 and Education_3 features, as we asked get_dummies to drop first in order to preserve memory.
- We split the dataset at 30%(1500 rows) test, 70%(3500 rows) for training.
- The percentage of classes in test shows that we have an imbalanced dataset, around 90% of the customers did not accept the loan up to now.

Model Building – Building steps of Decision Tree

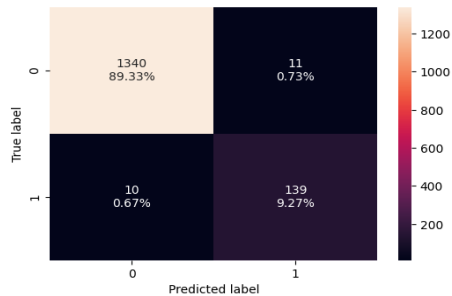
- Clean up the dataset, categorize the features properly, and hot zone data like zip code and education to improve prediction accuracy.
- Split the data into train and test.
- Create the empty decision tree.
- Fit the model to recursively build the tree structure.
 - Select the best split to reduce impurity
 - Split the dataset
 - Repeat the process until a node is pure or reaches max depth.

Model Building – Model Performance

- The first model is overfitting, 3169 TN and 331 TP, 0 FP and 0 FN, which leads to Accuracy, Recall, Precision, and F1 = 1. It gets 100 on the training data, which means it probably did not understand the problem. It may need pruning to get fixed.
- The confusion matrix of the test data shows 1340 TN, 139 TP, 10 FN, 11 FP. Recall shows that we caught 139 out of 149 borrowers ($139/(139+10) = 93\%$). We successfully identified 131 borrowers.
- The model looked perfect during training, with no FN or FP; the test data revealed that it was not true. We successfully captured 93% of the borrowers, but missed 07% (10 people). To capture these 10 customers, we need to prune the tree or adjust the class weight.
- Overall, the model performed excellently. It captures 93% of the **market** in the test set. For a bank trying to grow "Beyond 9%," this is an excellent result. There are only 10 missed opportunities.



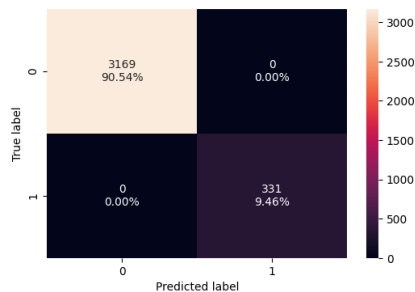
Training Data



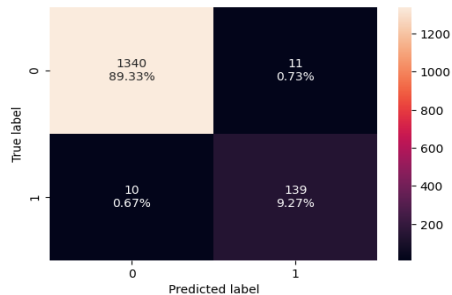
Test Data

Model Building – Model Performance pre-pruning

- In the pre-pruning phase, we iterate through several models based on max depth, max leaf nodes, and min samples to find the best well-balanced model. By checking the recall parameter of the models, it tries to determine, for the positive cases, the minimum possible miss.
- The code looks for the not too simple (underfitting), not too complex (overfitting), but just right.
- The results were pretty much the same of first option without pruning.
- To possibly improve even further, we may have to change min_samples_leaf, which can improve precision and reduce overfitting.



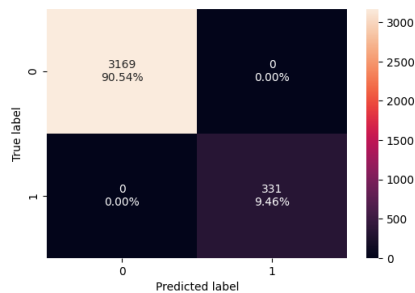
Training Data



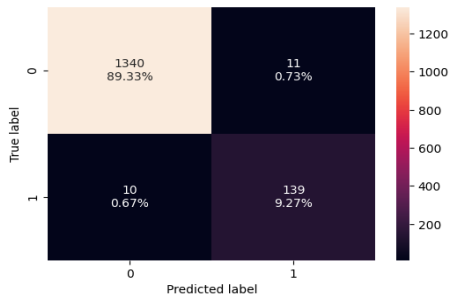
Test Data

Model Building – Model Performance post-pruning

- Post-pruning tends to be more accurate, keeps a simple tree that generalizes better, avoids premature stops, and results in lower test errors, without affecting the final results. It is particularly powerful when the data is noisy, because it reduces the noise.
- Post-pruning probably did not change the results because of the quality of the data and the strength of the features in the dataset.
- The results were pretty much the same of first option without pruning.
- To possibly improve even further, we may have to change min_samples_leaf, which can improve precision and reduce overfitting.



Training Data



Test Data

Model Performance Summary – Model Eval Criterion

The Confusion Matrix (The Foundation) provides the framework for evaluating and calculating Accuracy, Recall, Precision, and F1. These are the pillars to identify whether a model is achieving its goals.

All metrics are calculated from these four buckets:

True Positives (TP): Correctly identified Borrowers.

True Negatives (TN): Correctly ignored Non-Borrowers.

False Positives (FP): "False Alarm" (Contacted a non-interested customer).

False Negatives (FN): "Missed Opportunity" (Ignored a real borrower).

Model Performance Summary – Model Parameters

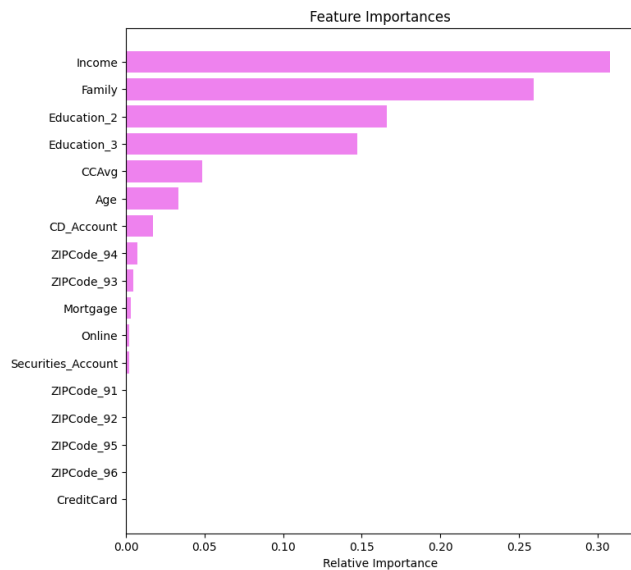
Parameters that affected the model performance.

- Default
 - Random_state
- Pre-Pruning
 - max_depth_values
 - max_leaf_nodes_values
 - min_samples_split_values
 - Class_weight
- Post-Pruning
 - CCP_alpha

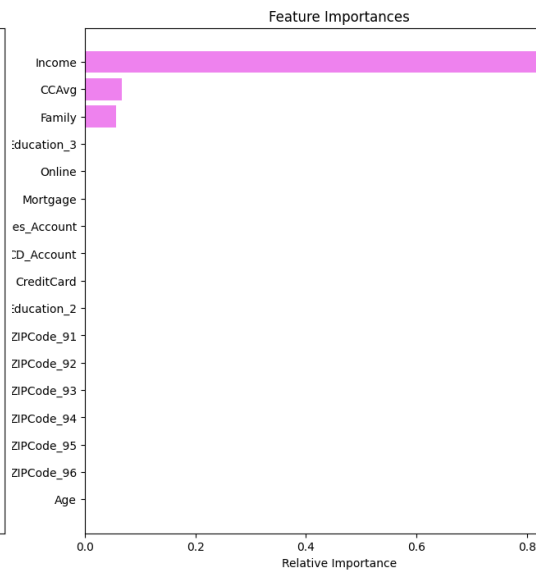
Model Performance Summary – Most important Features

In all the models exercised the most important variables were Income, Family, and CCAvg. A large family probably has special needs, making them a strong candidate for a loan.

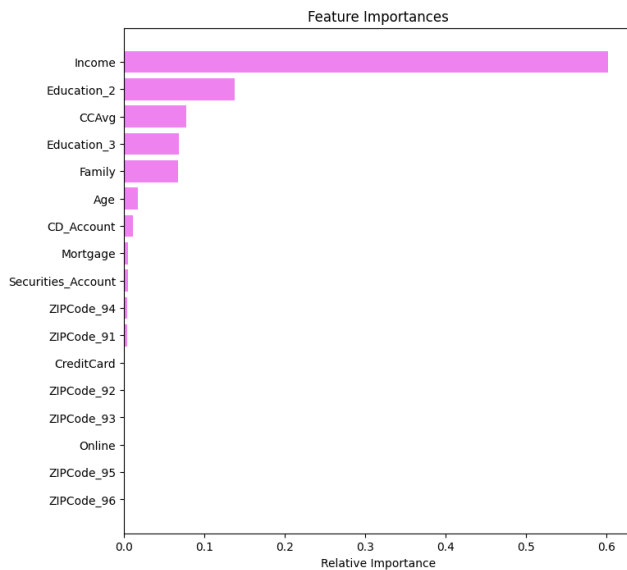
Default



Pre-Pruning



Post-Pruning



Model Performance Summary – Key Performance Metrics

	Decision Tree (sklearn default)	Decision Tree (Pre-Pruning)	Decision Tree (Post-Pruning)
Accuracy	0.986	0.986	0.986
Recall	0.932886	0.932886	0.932886
Precision	0.926667	0.926667	0.926667
F1	0.929766	0.929766	0.929766

The similar results show that the dataset had strong features(Income, Education, or CCAvg) that clearly distinguish borrowers from non-borrowers. The default model found the optimal path to separate customers based on the strong data.

We added extra parameters for pre-pruning and post-pruning, but the results were the same. The natural split was already the best split.

Model Performance Improvement

- Despite applying different model pruning methods (default, pre-pruning, post-pruning), the results ended up the same.
- The fact that the importance of the **features** changed drastically as we changed the pruning method did not affect the result. It shows that income, family, and average credit card spending were the strongest factors driving the decision.

APPENDIX

Data Background and Contents

The loan dataset provided was robust and high-quality, characterized by strong predictive features and minimal outliers. While the overall data integrity was excellent, we did identify and address latent missing values within the 'Experience' feature.

The dataset

- ID: Customer ID
- Age: Customer's age in completed years
- Experience: # years of professional experience
- Income: Annual income of the customer (in thousand dollars)
- ZIP Code: Home Address ZIP code.
- Family: The family size of the customer
- CCAvg: Average spending on credit cards per month (in thousand dollars)
- Education: Education Level. 1: Undergrad; 2: Graduate; 3: Advanced/Professional
- Mortgage: Value of house mortgage if any. (in thousand dollars)
- Personal_Loan: Did this customer accept the personal loan offered in the last campaign?
- Securities_Account: Does the customer have a securities account with the bank?
- CD_Account: Does the customer have a certificate of deposit (CD) account with the bank?
- Online: Do customers use Internet banking facilities?
- CreditCard: Does the customer use a credit card issued by any other Bank (excluding All Life Bank)?

Slide Header

- Please add any other pointers (if needed)



Happy Learning !

